



PUC Minas
Virtual

Práticas Técnicas Básicas DevOps

Marco Mendes

Afinal, o que é DevOps?



O que é o DevOps?

- a) *Pipelines* para construir e implantar código em ambiente de produção
- b) Um conjunto de ferramentas para acelerar o desenvolvimento de aplicações
- c) Uma metodologia

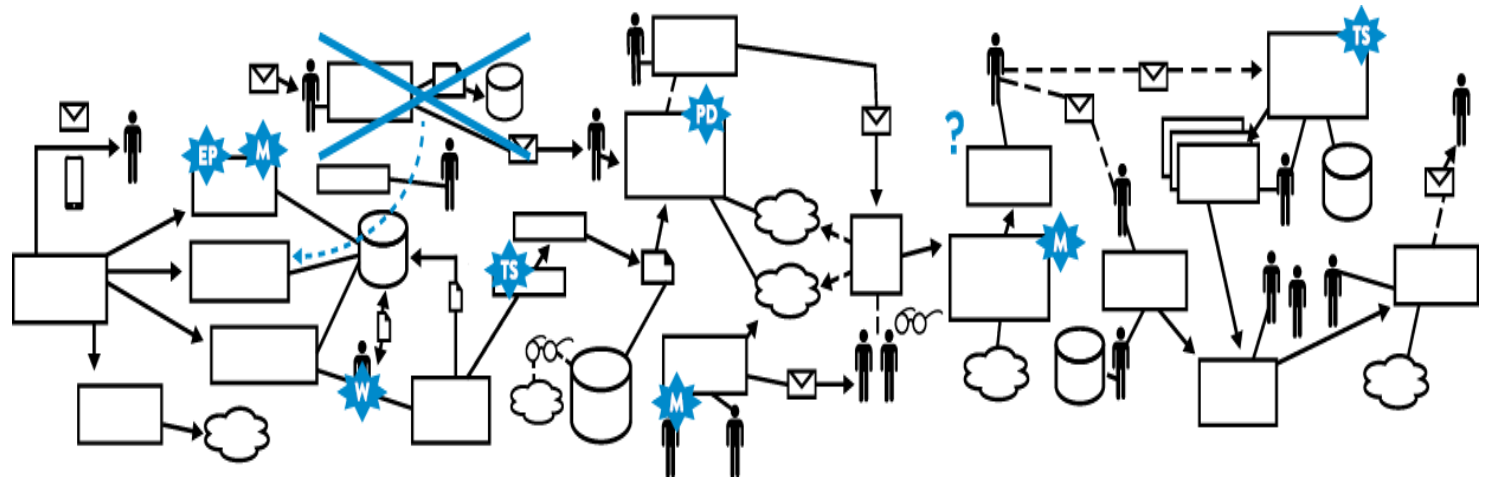


O que é o DevOps?

Um *abordagem* baseada em **capacidades técnicas, de processo e culturais** que impulsionam uma maior entrega de software e desempenho organizacional, melhorando a colaboração entre times de desenvolvimento, segurança, qualidade e infraestrutura.



O por que do DevOps?



*Fonte: Um fluxo de valor de tecnologia com um **tempo de entrega** de vários meses (Fonte: Damon Edwards, “DevOps Kaizen,” 2015.)*

Os caminhos do DevOps



O primeiro caminho permite um fluxo rápido de trabalho da esquerda para a direita de Desenvolvimento para Operações para o cliente.



O segundo caminho permite o fluxo rápido e constante de feedback da direita para a esquerda em todos os estágios do fluxo de valor.



O terceiro caminho possibilita a criação de uma cultura generativa de alta confiança que apoie uma abordagem dinâmica, disciplinada e científica à experimentação e tomada de riscos, facilitando a criação de aprendizado organizacional, tanto de nossos sucessos como fracassos.



O primeiro caminho permite um fluxo rápido de trabalho da esquerda para a direita de Desenvolvimento para Operações para o cliente.

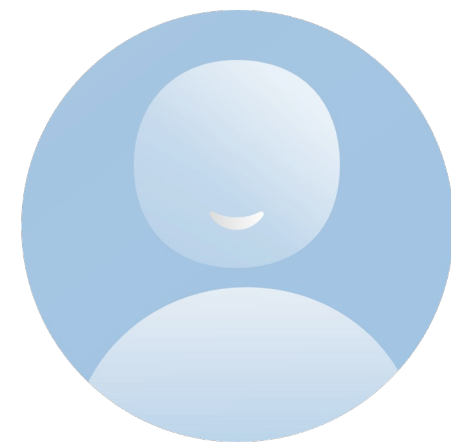


O segundo caminho
permite o fluxo rápido e
constante de feedback da
direita para a esquerda
em todos os estágios do
fluxo de valor.



O terceiro caminho possibilita a criação de uma cultura generativa de alta confiança que apoie uma abordagem dinâmica, disciplinada e científica à experimentação e tomada de riscos, facilitando a criação de aprendizado organizacional, tanto de nossos sucessos como fracassos.

Capacidades Técnicas Básicas DevOps



Gestão de Configuração de Código



Prática para gerir código fonte, arquivos de infraestrutura como código, para controle de versionamento, linhas de base e rastreabilidade.

Automação de Testes



Prática para a execução automatizada de testes em diferentes níveis e objetivos.



Prática para construção
automatizada de
executáveis robustos
(*builds*).

Entrega Contínua



Prática para a implantação segura de executáveis em ambientes de homologação e produção (*releases*).

Infraestrutura como Código



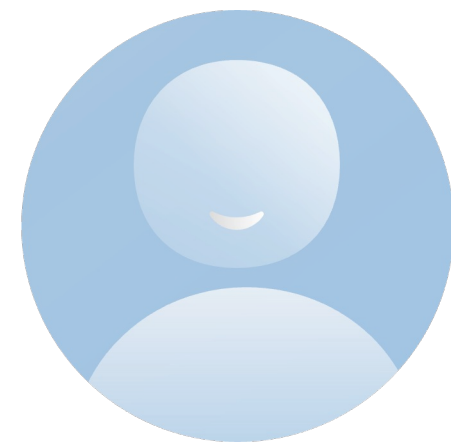
Prática para executar
provisionar ativos de
infraestrutura e nuvens
através de código.

Containerização



Prática para virtualização de ambientes, servidores e software através de contêineres leves.

GitHub e GitHub Actions



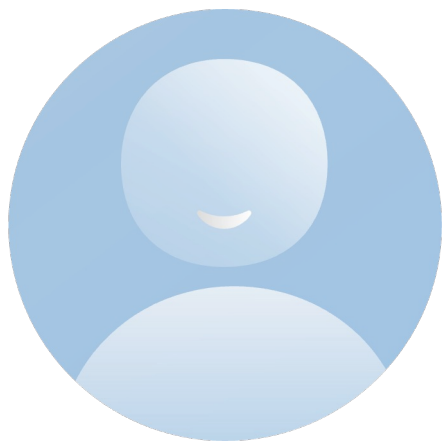
***Repositório Git de nuvem e um
provedor de serviços***

***Gestão e armazenamento de
repositórios Git***

***Armazenamento
push/pull***

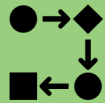
***Automação e CI/CD
(GitHub Actions)***

***Gestão colaborativa de
código (pull requests)***



GitHub Actions – Elementos Chave

Workflows



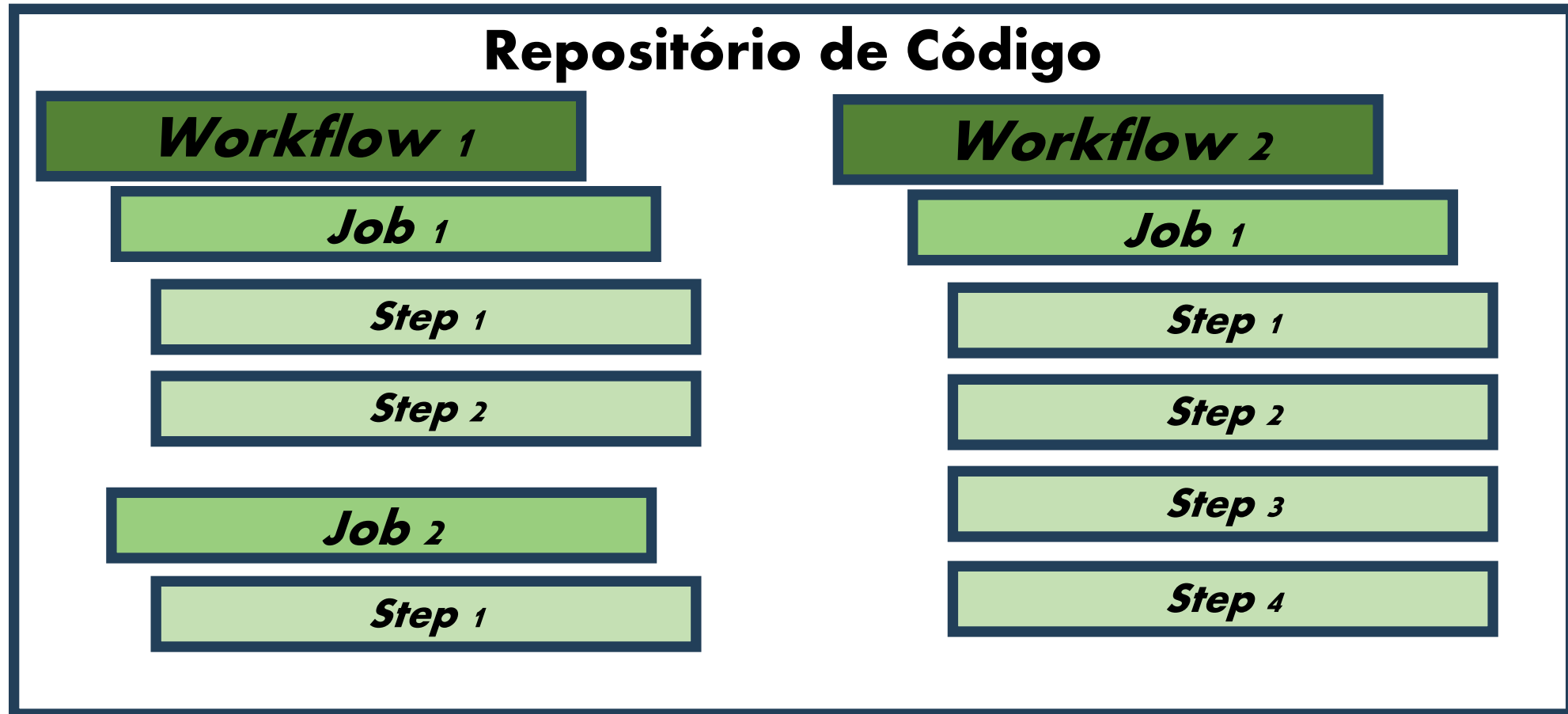
Jobs



Steps

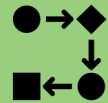


GitHub Actions – Elementos Chave



GitHub Actions – Elementos Chave

Workflows



Vinculado a um repositório GitHub

Contém um ou mais Jobs

Ativado através de eventos

Jobs



Define o ambiente de execução (Runner)

Contém um ou mais passos

*Roda em paralelo ou sequencialmente.
Pode ser condicional.*

Steps



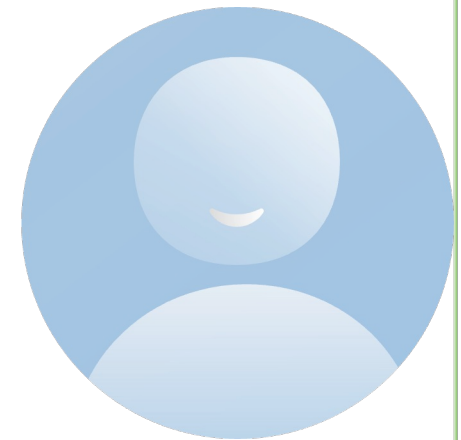
Executa um script de linha de comando (Shell script) ou uma ação (Action)

Pode executar ações de terceiros ou personalizadas

Passos são executados em ordem. Podem ser condicionais

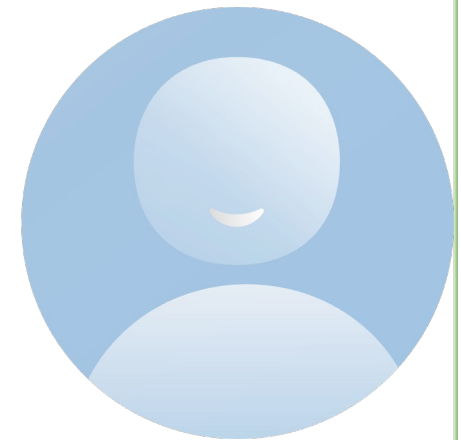
Primeiro Workflow

GitHub Actions



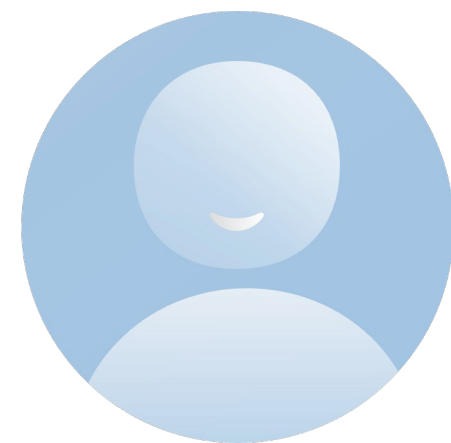
Workflow Básico

GitHub Actions



Gerenciamento de Falhas

GitHub Actions



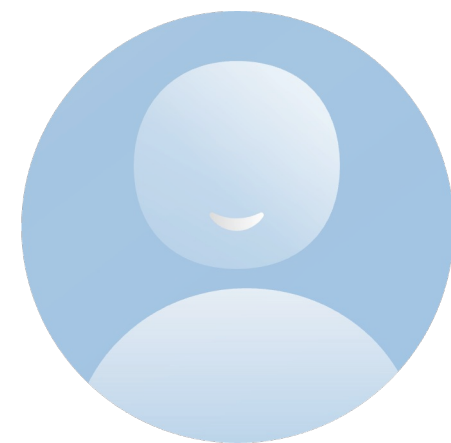
Execução de Múltiplos Jobs

GitHub Actions

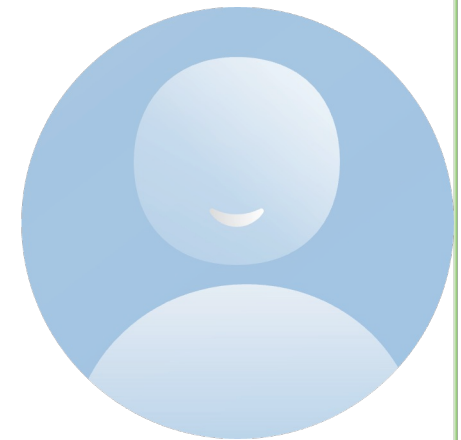


Geração de Artefatos

GitHub Actions



Pipelines DevOps

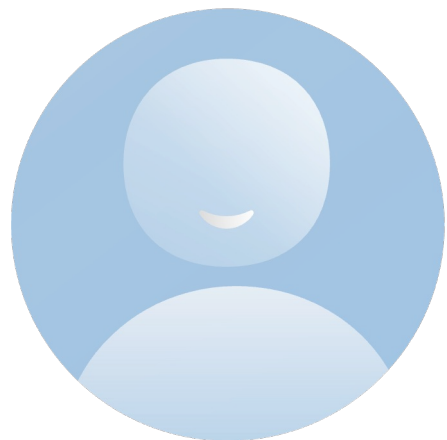


Pipelines DevOps

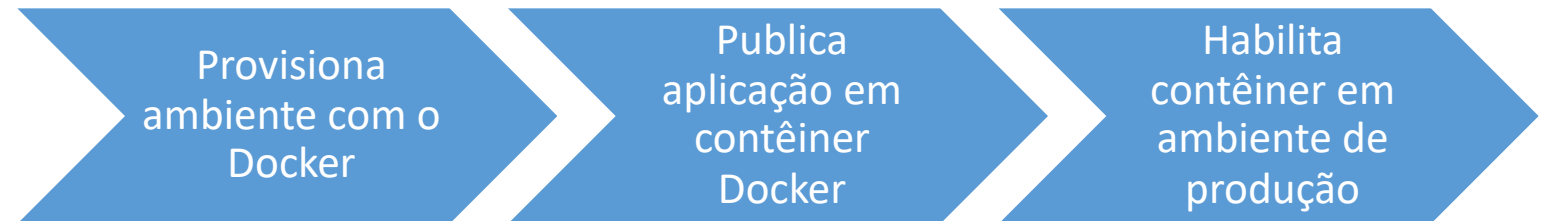
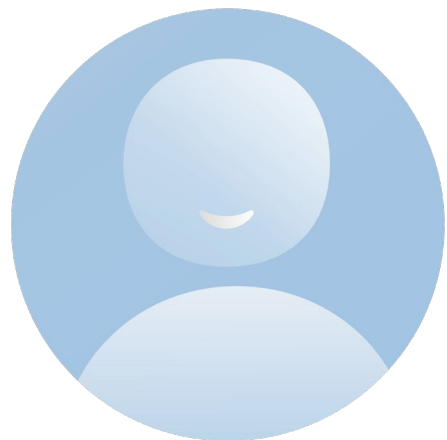


Mecanismos automatizados para reduzir o **tempo de entrega** de demandas, aumentar o **feedback** entre pessoas e também habilitar **experimentos e inovações**.

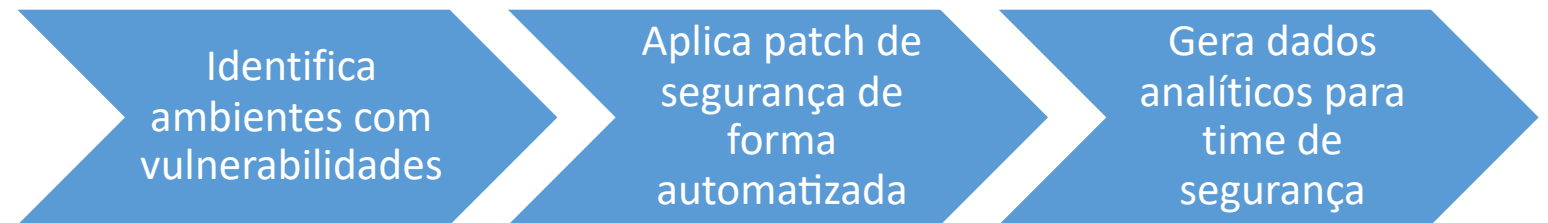
Pipeline de Integração Contínua (CI)



Pipeline de Entrega Contínua (CD)



Pipeline de Segurança de Containeres

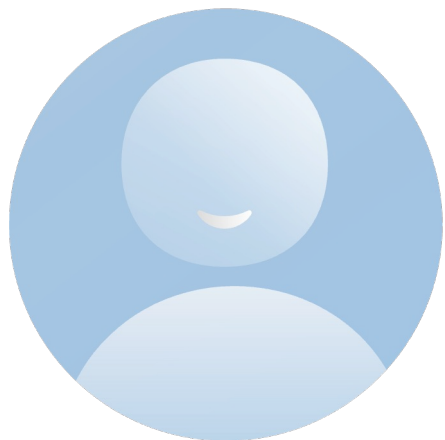


Pipelines DevOps



As linhas de montagem DevOps estão focadas em automatizar e conectar atividades realizadas por várias equipes

Exemplos



Aprovação de código fonte em troncos específicos

Builds para desenvolvedores

Provisionamento de infraestrutura

Gerenciamento de configuração para operações

Teste de automação de performance de aplicações,

Detecção de vulnerabilidade de segurança

Benefícios

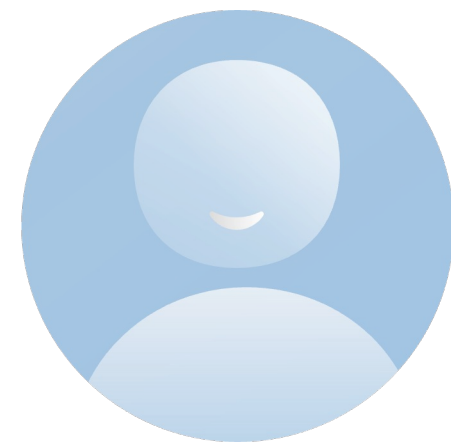
Redução do tempo de execução para executar as tarefas nos centros de trabalho

Capacidade de definir facilmente fluxos de trabalho em vários pipelines

Automação da infraestrutura de DevOps, incluindo containers e ambientes de nuvens

Capacidade de transmitir estado e outras informações (*feedback* e *feedforward*)

Automação de *Builds* e CI



Times de baixa maturidade DevOps



Passam a maior parte do tempo de um projeto com o software sem funcionar.

Operam durante muito tempo com códigos isolados.

Adiam tarefas de integração e testes de aceite para o final do projeto.

O que é a automação de builds?

- A **automação de builds** é prática essencial para garantir que os executáveis dos seus produtos sejam gerados de forma consistente, em base diária.
- Esta prática busca evitar o problema comum do código funcionar apenas máquina do desenvolvedor.

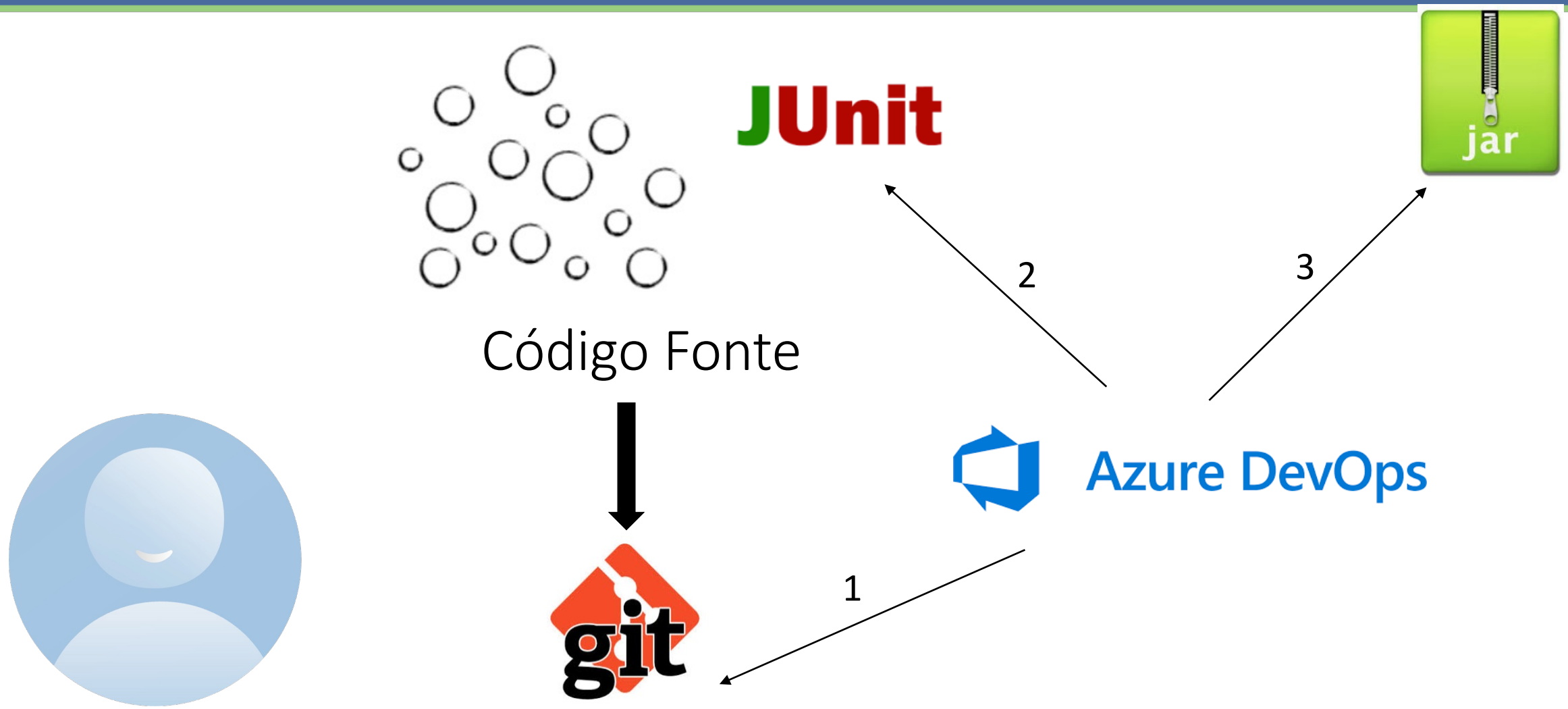


Como a automação de builds funciona?

- A automação de builds externaliza todas as dependências de bibliotecas e configurações feitas dentro de uma IDE para scripts e que possa ser consistentemente executado por robôs.
- Ela é pilar básico para avançarmos em direção a fluxos de CI



Caso real

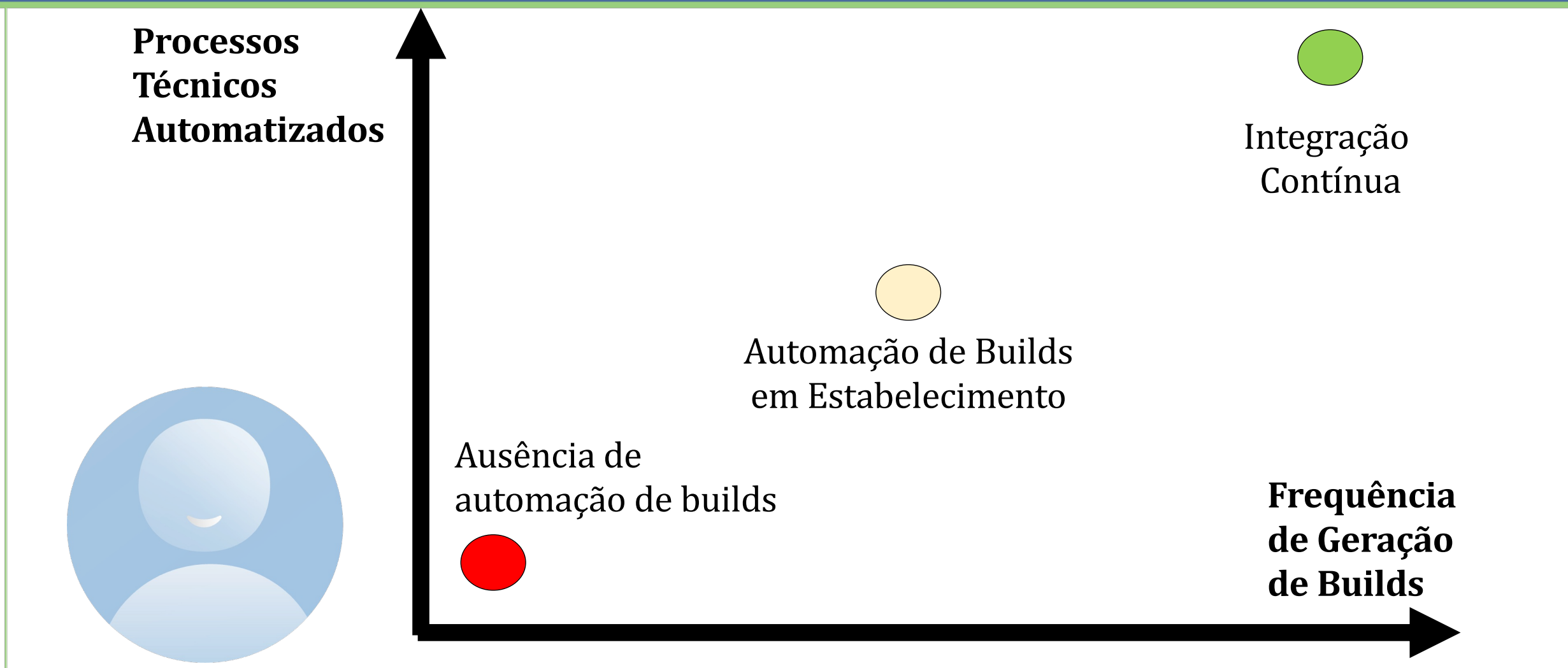


A barra mínima para automação de builds

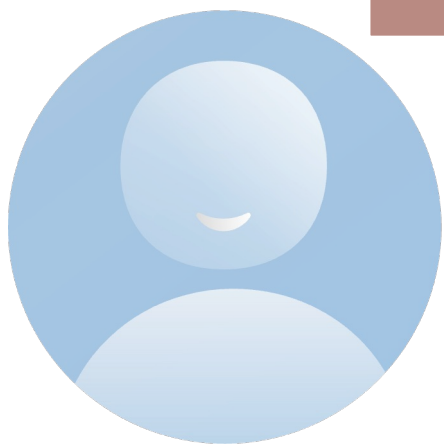
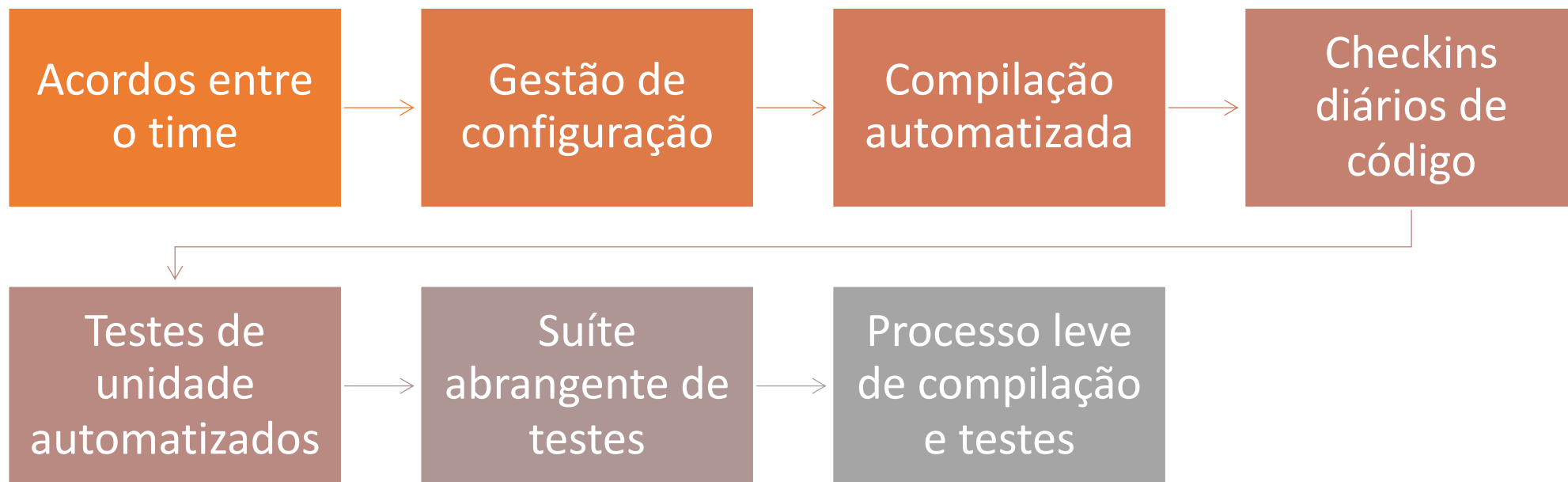
Embora a automação de builds, na sua definição inicial, lide apenas com a construção de um build, a prática comum de mercado é que builds devam executar um **conjunto mínimo de testes de unidade automatizados (smoke tests)** para estabelecerem confiabilidade mínima ao executável sendo produzido.



Automação de Builds versus CI

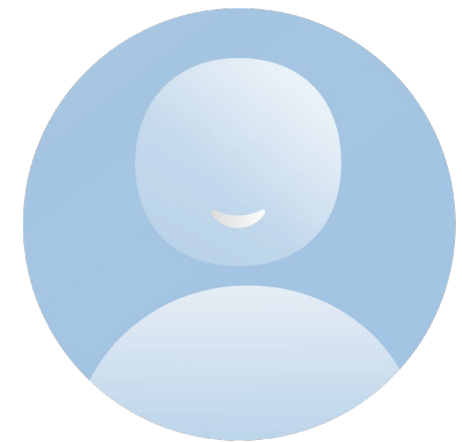


Requisitos



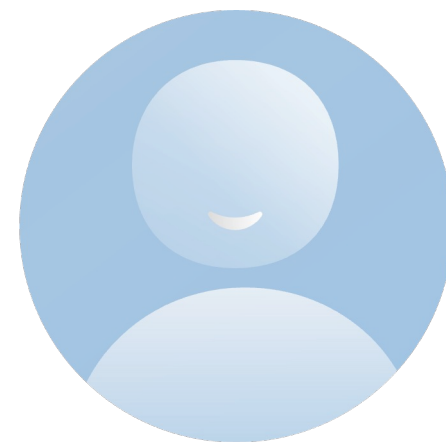
Caso Real com o Jenkins em 2012 - Fracasso

- Inovação trazida de forma unilateral pelo “cara mais esperto na sala”
- Time de desenvolvimento ainda sem maturidade em gestão de configuração e automação de testes.
- Esforços de builds automatizados fracassaram e eventualmente foram abortados pelo gestor.

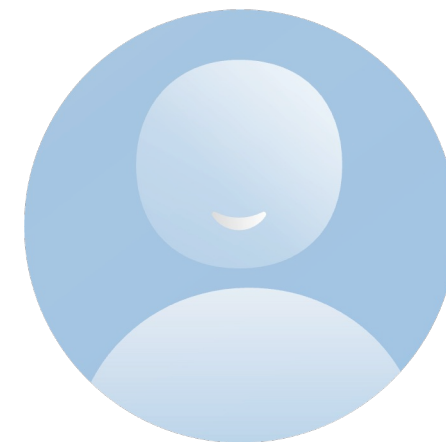


Caso Real com o Jenkins em 2015 - Sucesso

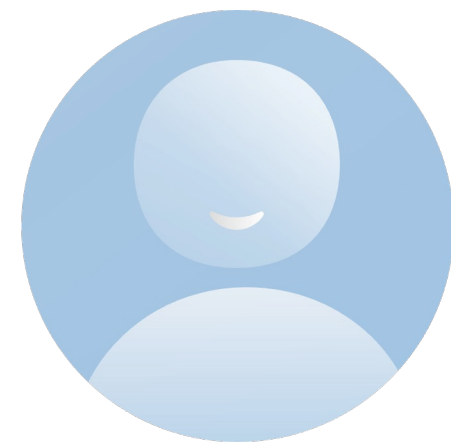
- Testes de unidades foram estabelecidos e aprimorados como prática do time.
- Durante 9 meses, o time de quase 15 pessoas cresceu a linha base de testes de unidade para quase 5000 testes automatizados.
- Pipelines DevOps com automação de builds foram desenvolvidos e se expandiram para rodar em troncos específicos na gestão de configuração, em base diária e depois em múltiplas vezes por dia.



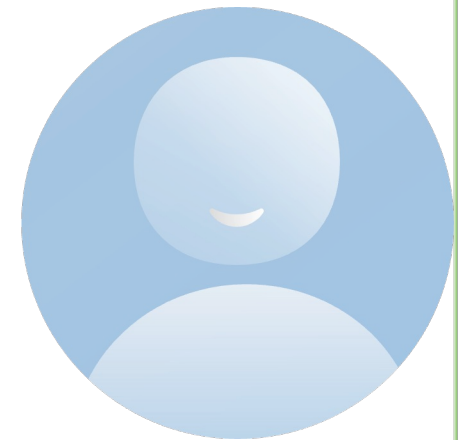
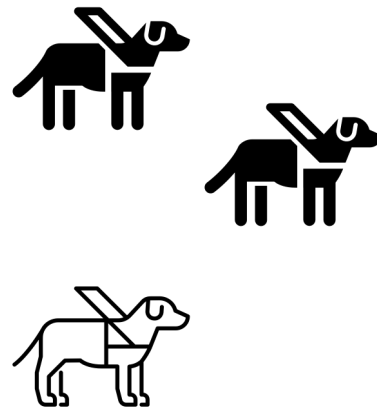
Ferramentas CNCF



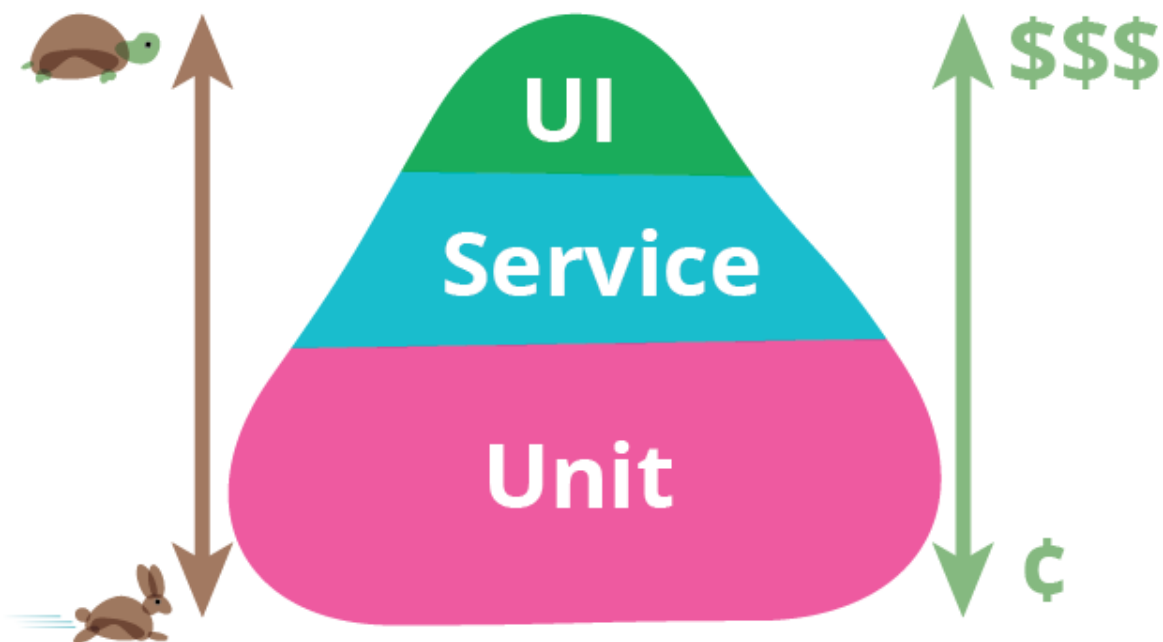
Automação de Testes



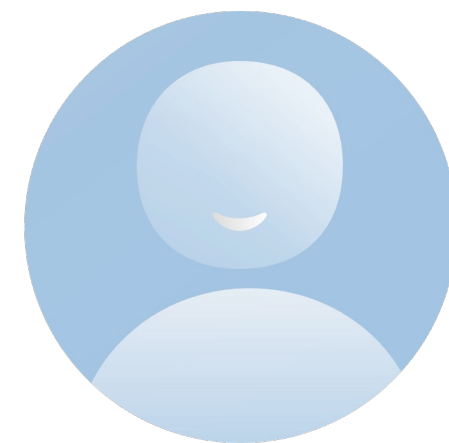
Automação de testes é peça fundamental nos processos DevOps



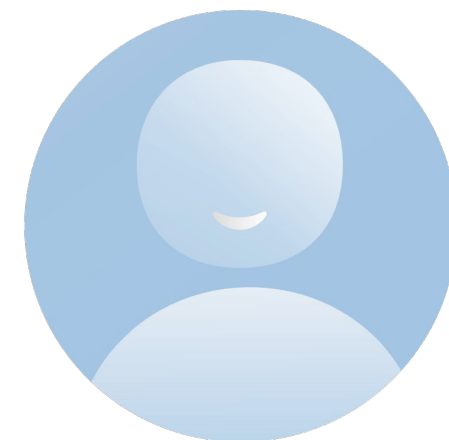
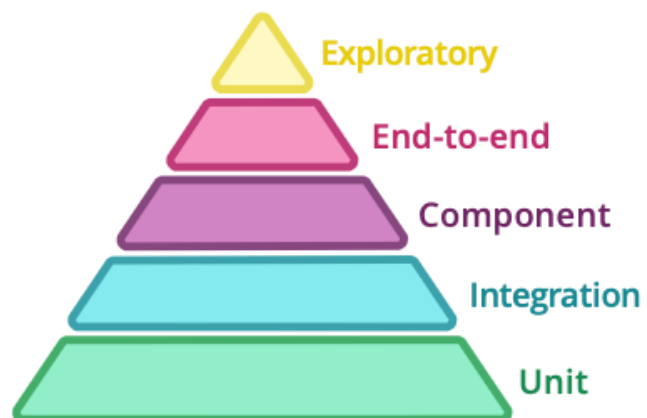
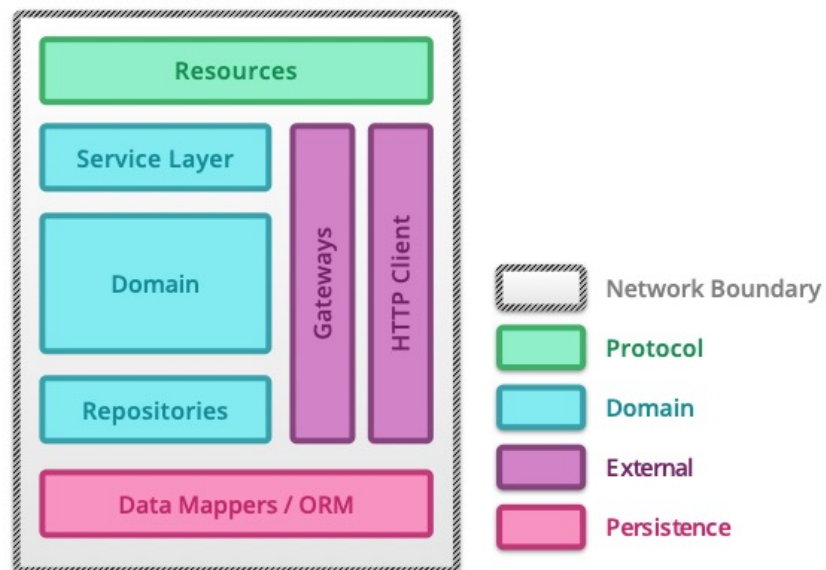
O que deve ser automatizado?



Fonte: martinfowler.com/bliki/TestPyramid.html



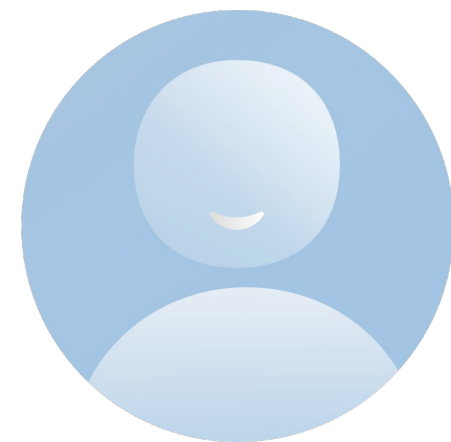
O que deve ser automatizado?



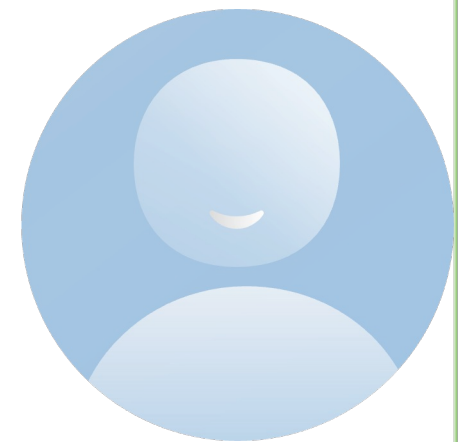
Fonte: martinfowler.com/bliki/TestPyramid.html

Testes em produção

- Ambientes produtivos podem ter pipelines para proteção de novas publicações.
- Tipicamente testes preparados para leituras e/ou gravações que não irão gerar impacto em dados reais.
- Chamados de “smoke tests”



Automação de Entregas



O que é automatizar entregas?



A automação das entregas é uma prática que busca garantir que o processo de promoção do executável para os ambientes de testes, homologação e produção sejam automatizados e assim tornados consistentes e eficientes.

Benefícios



reduzir o tempo para entregar um novo *build* em ambiente de produção através da automação da instalação e configuração de ferramentas e componentes arquiteturais;

reduzir erros em implantação causadas por parâmetros específicos que não foram corretamente configurados pelos times de desenvolvimento e operações

minimizar a fricção entre os times de desenvolvimento, QA e operações

prover confiabilidade e segurança no processo de implantar aplicações

Pontos de atenção



A automação de entregas não deve “compilar” o código fonte. Ela deve promover o builds com a reconfiguração variáveis de ambientes e outras informações de parâmetros conforme o desejo.

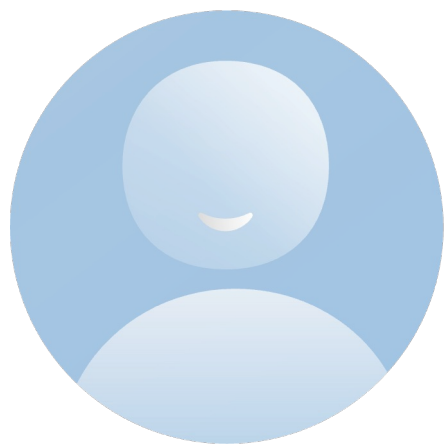
Pontos de atenção



A automação de entregas pode incluir solicitações de aprovação pré-implantação e pós-implantação.

Isso automatiza processos de gestão de mudanças em organizações que usem ITIL.

Caso Real



Azure DevOps

Builds são coletados de
ambientes de integração
(*staging areas*)

1



Promoção para homologação e
aprovação pós-publicação
pelo time de testes.

2



3



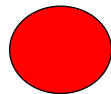
Aprovação pré-publicação e promoção
para ambiente de produção pelo gestor.

Automação de Entregas vs CD

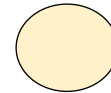
**Processos
Técnicos
Automatizados**



Promoção de builds
manual e demorada



Promoção de Builds
em Estabelecimento



Entrega Contínua (CD)



**Frequência
de Promoção
de Builds**



Ferramentas para CD



Entrega Contínua versus Implantação Contínua



Continuous Delivery (Entrega contínua)



A entrega contínua realiza a promoção automatizada entre ambientes de desenvolvimento.

Exemplos: ambientes de testes integrado ou ambientes de homologação

Continuous Delivery (Entrega contínua)



Na entrega contínua, entretanto, a etapa de publicação para o ambiente de produção requer aprovação humana, em conformidade por exemplo com regulatórios exigidos por ITIL.

Continuous Deployment (Implantação contínua)

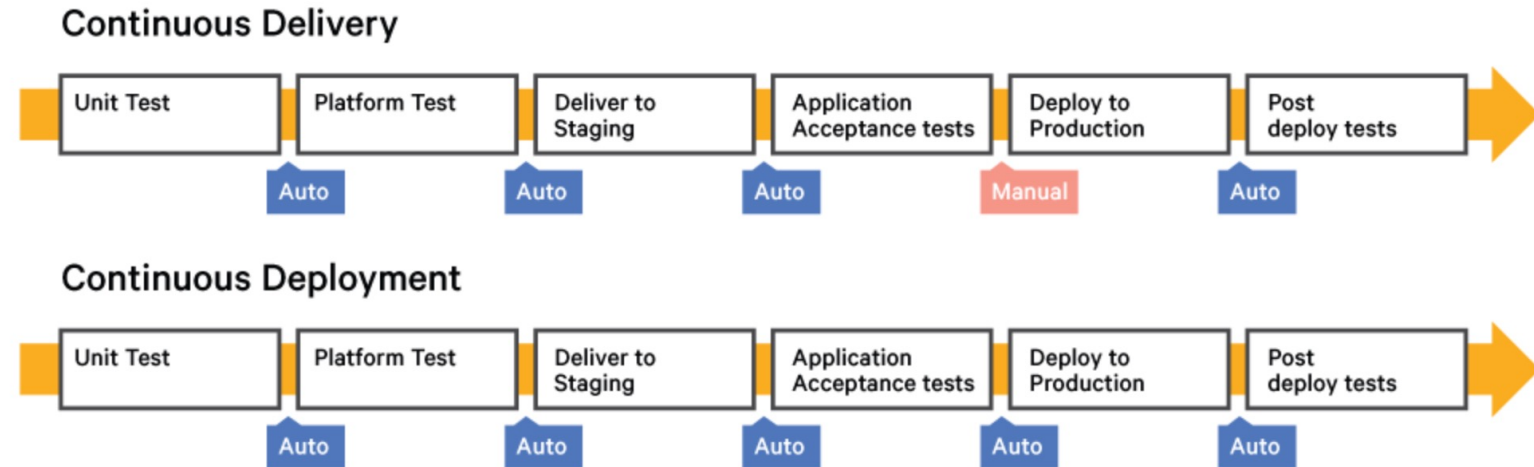


Com a entrega contínua implantada, é possível automatizar até mesmo a transferência de objetos para a produção.

Isso é desejado em ambientes altamente dinâmicos como startups e aplicações que não estejam submetidas a elementos regulatórios de ITIL.

Ou organizações que possuem implementações avançadas de gestão de mudanças no ITIL.

Continuous Deployment (Implantação contínua)



Fonte: puppet.com/blog/continuous-delivery-vs-continuous-deployment-what-s-diff



Pontos de Atenção



Automatizar a promoção de builds também deve considerar a promoção de scripts de banco de dados e até mesmo o transporte de dados de configuração.

Ignorar o processo de automação nas bases de dados criam processos frágeis de entrega contínua.

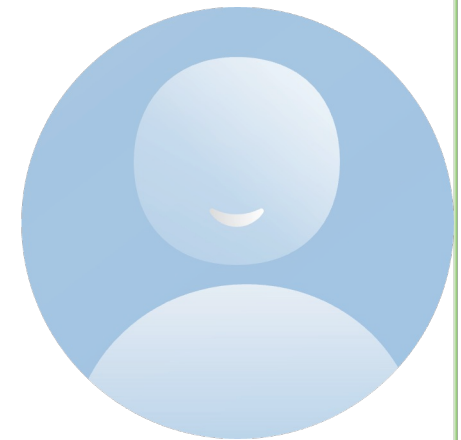
Pontos de Atenção



Toda promoção de ambientes deve incluir processos de verificação da estabilidade.

Testes pós-implantação e testes de fumaça são práticas comuns para criar pipelines de entrega robustos.

Observabilidade e Análise



O que é a Observabilidade?



O processo de administração de ambientes de produção tem se automatizado.

As tecnologias para monitoração, registro (logging) e rastreamento permitem observar e analisar eventos diversos em ambientes de software, banco de dados, infraestrutura, eventos de negócio.

Análise de tendências, correlações complexas e capacidade de antecipação também tem sido incorporadas a tecnologias de análise de ambientes.

O que é a Observabilidade?



Processos de observabilidade nascem ainda com os times de desenvolvimento.

Uma boa arquitetura técnica para geração de registros (logs) é essencial para facilitar a análise de problemas e incidentes em produção.

O que é a Observabilidade?



Processos de observabilidade nascem ainda com os times de desenvolvimento.

Uma boa arquitetura técnica para geração de registros (logs) é essencial para facilitar a análise de problemas e incidentes em produção.

Níveis de observabilidade



Monitoramento

Instrumentação de um aplicativo para coletar, agregar e analisar registros e métricas para melhorar nossa compreensão de seu comportamento.

O monitoramento inclui tudo, desde observar o espaço em disco, o uso da CPU e o consumo de memória em nós individuais até fazer transações sintéticas detalhadas para ver se um sistema ou aplicativo está respondendo corretamente e em tempo hábil.

PUC Minas Virtual

Registro (log)

Os aplicativos emitem um fluxo constante de mensagens de log descrevendo o que estão fazendo a qualquer momento.

Essas mensagens de log capturam vários eventos acontecendo no sistema, como ações com falha ou bem-sucedidas, informações de auditoria ou eventos de saúde.

As ferramentas de registro coletam, armazenam e analisam essas mensagens para rastrear relatórios de erros e dados relacionados.

PUC Minas Virtual

Rastreamento

Em um mundo de microsserviços, os serviços estão constantemente se comunicando uns com os outros pela rede.

O rastreamento, um uso especializado de registro, permite rastrear o caminho de uma solicitação à medida que ela se move através de um sistema distribuído.

PUC Minas Virtual

Monitoramento



Instrumentação de um aplicativo para coletar, agregar e analisar registros e métricas para melhorar nossa compreensão de seu comportamento.

O monitoramento inclui tudo, desde observar o espaço em disco, o uso da CPU e o consumo de memória em nós individuais até fazer transações sintéticas detalhadas para ver se um sistema ou aplicativo está respondendo corretamente e em tempo hábil.

Registro (log)



Os aplicativos emitem um fluxo constante de mensagens de log descrevendo o que estão fazendo a qualquer momento.

Essas mensagens de log capturam vários eventos acontecendo no sistema, como ações com falha ou bem-sucedidas, informações de auditoria ou eventos de saúde.

As ferramentas de registro coletam, armazenam e analisam essas mensagens para rastrear relatórios de erros e dados relacionados.

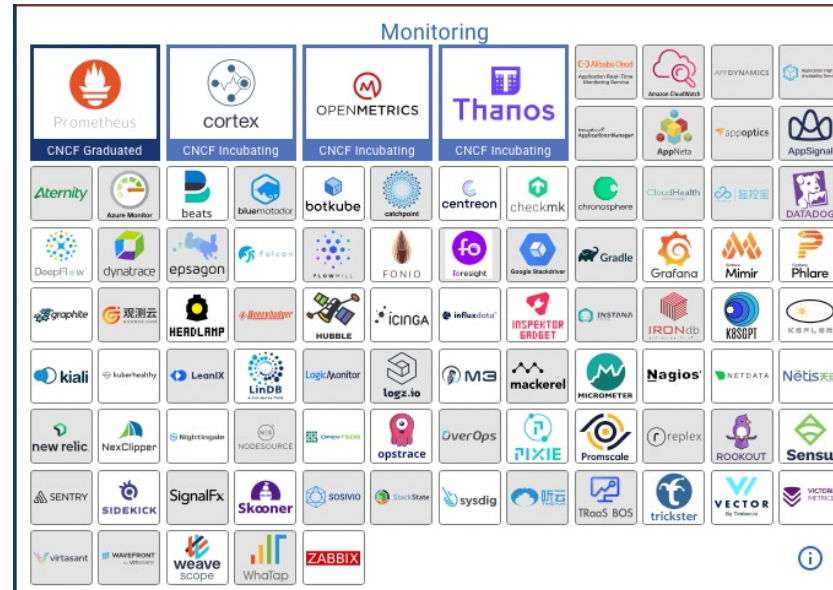
Rastreamento



Em um mundo de microsserviços, os serviços estão constantemente se comunicando uns com os outros pela rede.

O rastreamento, um uso especializado de registro, permite rastrear o caminho de uma solicitação à medida que ela se move através de um sistema distribuído.

Visão CNCF para Observabilidade





PUC Minas
Virtual